MODULE-II

DIVIDE AND CONQUER METHOD

TOPICS

- Control Abstraction for Divide and Conquer
- Problems-Solved using Divide and Conquer Technique
    a. 2- way Merge Sort
    b. Quick sort
    c. Binary Search
    d. Finding Maximum and minimum,
    e. Divide and Conquer-Matrix Multiplication.(Strassen's Matrix Multiplication)

DIVIDE AND CONQUER METHOD

- ✓ A divide-and-conquer algorithm works by recursively breaking down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly.
- ✓ The solutions to the sub-problems are then combined to give a solution to the original problem.
- ✓ This technique can be divided into the following three parts:

I. **Divide:** This involves dividing the problem into some sub problem.

II. **Conquer:** Solve the Sub problem by calling recursively until all the sub problems solved.

III. **Combine:** The Sub problem solved so that we will get find problem solution.

Divide/Break

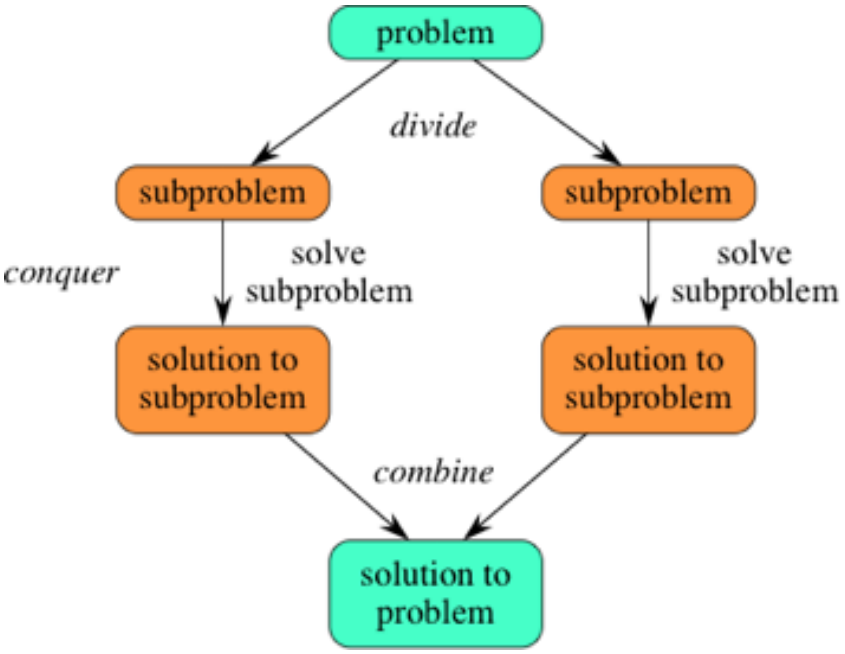
- This step involves breaking the problem into smaller sub-problems.
- Sub-problems should represent a part of the original problem.
- This step generally takes a recursive approach to divide the problem until no sub-problem is further divisible.
- At this stage, sub-problems become atomic in nature but still represent some part of the actual problem.

Conquer/Solve

- This step receives a lot of smaller sub-problems to be solved.
- Generally, at this level, the problems are considered 'solved' on their own.

Merge/Combine

- When the smaller sub-problems are solved, this stage recursively combines them until they formulate a solution of the original problem.



Control Abstraction of Divide and Conquer

- A control abstraction is a procedure whose flow of control is clear but whose primary operations are specified by other procedures whose precise meanings are left undefined.
- The control abstraction for divide and conquer technique is DANDC(P), where P is the problem to be solved.

```
DANDC (P)
{
if SMALL (P) then return S (P);
else
{
Divide P into smaller instances P1, P2, … . Pk, k≥1
Apply DANDC to each of these sub problems;
Return (COMBINE (DANDC (P1), DANDC (P2),… ., DANDC (Pk)) ;
}
}
```

✓ SMALL(P) is a Boolean valued function which determines

whether the input size is small enough so that the answer can be computed without splitting.

- ✓ If this is so function 'S' is invoked otherwise, the problem 'P' is divided into smaller sub problems.
- ✓ These sub problems P_1, P_2, . . . , P_k are solved by recursive application of DANDC.

2-way Merge Sort

- We're using divide-and-conquer to sort; we need to decide what our sub problems are going to look like.
- The full problem is to sort an entire array. Let's say that a sub problem is to sort a sub array.
- We'll think of a subproblem as sorting the subarray starting at index p and going through index r.
- In terms of our notation, for an array of n elements, we can say that the original problem is to sort array [0..n-1].

How merge sort uses divide-and-conquer:

1. **Divide** by finding the number q of the position midway between p and r.
2. **Conquer** by recursively sorting the subarrays in each of the two subproblems created by the divide step. That is, recursively sort the subarray array [p..q] and recursively sort the subarray array [q+1..r].
3. **Combine** by merging the two sorted subarrays back into the single

sorted subarray array [p..r].

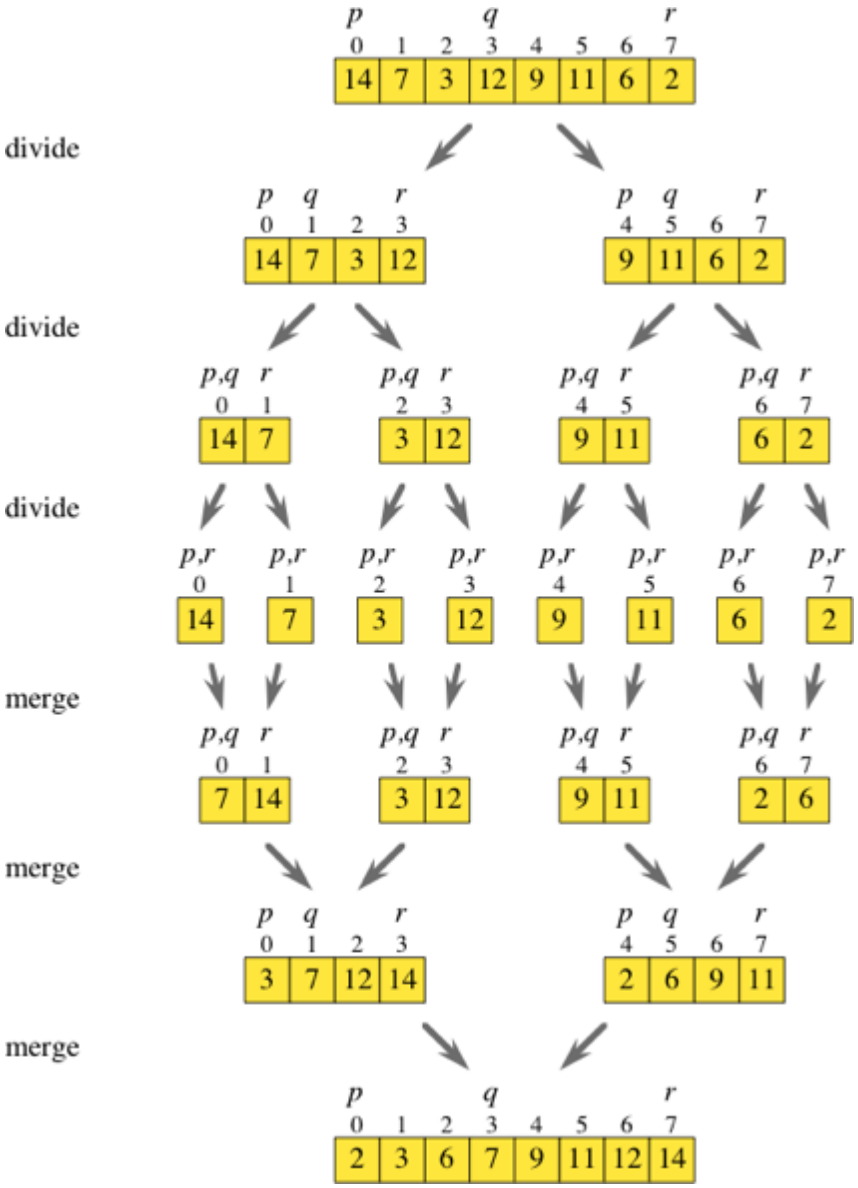
Example

Let's start with array holding **[14, 7, 3, 12, 9, 11, 6, 2],** so that the first subarray is actually the full array, **array [0..7] (p=0, and r=7).**

a. In the **divide** step, we compute q=3.
b. The **conquer** step has us sort the two subarrays **array [0..3],** which contains [**14, 7, 3, 12],** and **array [4..7],** which contains **[9, 11, 6, 2].**
    - When we come back from the conquer step, each of the two subarrays is sorted:

      array [0..3] contains [3, 7, 12, 14]
      array [4..7] contains [2, 6, 9, 11]

c. Finally, the combine step merges the two sorted subarrays in the first half and the second half, producing the
   Final sorted array [2, 3, 6, 7, 9, 11, 12, 14].
   WORKING OF MERGE SORT



Algorithm

Algorithm MERGESORT (low, high)

```
// a (low: high) is a global array to be sorted.
{
if (low < high)
{
mid := (low + high)/2 //finds where to split the set

MERGESORT(low, mid) //sort one subset

MERGESORT(mid+1, high) //sort the other subset

MERGE(low, mid, high) // combine the results
}

}
```

Algorithm MERGE (low, mid, high)

```
// a (low : high) is a global array containing two sorted subsets
// in a (low : mid) and in a (mid + 1 : high).
// The objective is to merge these sorted sets into single sorted
// set residing in a (low : high). An auxiliary array B is used.

{

h :=low; i := low; j:= mid + 1;

 while ((h < mid) and (J < high)) do

{

if (a[h] < a[j]) then

{

b[i] := a[h]; h := h + 1;

}
else
{

b[i] :=a[j]; j := j + 1;

}

i := i + 1;

}

if (h > mid) then

for k := j to high do

{

b[i] := a[k]; i := i + 1;

}
```

```
else
for k := h to mid do
{
b[i] := a[K]; i := i + l;
}
for k := low to high do
a[k] := b[k];
}
```